



Politechnika Łódzka
Instytut Automatyki

Laboratorium Robotów Mobilnych

Instrukcja 2

Łódź, 2019

Wstęp

Celem ćwiczenia jest zapoznanie się z podstawowymi instrukcjami Pythona, a także napisanie własnego Publisher'a i Subscriber'a wykorzystywanego przez ROS do przekazywania informacji. Instrukcja dotycząca Pythona i jego podstawowych instrukcji znajduje się w katalogu Python w pliku "Python wprowadzenie", natomiast plik "Python zadania" zawiera zadania do wykonania.

Tworzenie Publisher'a i Subscriber'a

Przed uruchomieniem poniższych skryptów wymagane jest uruchomienie ROS Mastera. W tym celu w terminalu wpisujemy:

```
roscore
```

Do swojego katalogu należy skopiować pliki i foldery z katalogu cw2. Następnie należy uruchomić kolejny terminal i przejść do swojego katalogu. W terminalu należy wpisać komendę otwierającą w przeglądarce edytor umożliwiający m.in. na tworzenie kodu w Pythonie.

```
jupyter notebook
```

W przeglądarce powinno pojawić się okno:



Z poziomu tego widoku możliwe jest uruchomienie już istniejących skryptów zapisanych w pliku jupyter'a (.jpynb) lub utworzenie nowego skryptu.

Do uruchomienia niektórych przykładowych skryptów jak i wykonania zadań konieczne jest uruchomienie następującego node'a `turtlesim_node` z paczki `turtlesim`.

```
roslun turtlesim turtlesim_node
```

Używanie wiadomości ROS

Przykładowa struktura wiadomości typu `WayPoint` z paczki `geographic_msgs`:

```
uuid_msgs/UniqueID id
uint8[16] uuid
geographic_msgs/GeoPoint position
float64 latitude
float64 longitude
float64 altitude
geographic_msgs/KeyValue[] props
string key
string value
```

Pierwszym elementem jest typ pola o podanej na drugim miejscu nazwie.

Importowanie wiadomości:

```
from geographic_msgs.msg import WayPoint
```

Inicjalizacja nowej wiadomości ROS'owej:

```
zmienna = WayPoint()
```

Dostanie się do danego pola zmiennej w celu ustawienia lub odczytania wartości. Np. aby ustawić wartość szerokości geograficznej *latitude* dla powyższej wiadomości należy:

```
zmienna.position.latitude = 17.293847
```

Nowy Publisher

Zapoznać się z kodem przedstawionym w poniższych przykładach odpowiadającym za stworzenie nowego Publisher'a. Poniższe przykładowe programy dostępne są w folderze ROS_example (pliki pub_example_1.ipynb oraz pub_example_2.ipynb).

```
1 #!/usr/bin/env python
2 #Powyzsza linia informuje, ze skrypt wykonywany jest jako skrypt Python'a
3
4 # Biblioteka umozliwiajaca tworzenie ROS Node i korzystanie z ROS
5 import rospy
6
7 # Import wiadomosci rosowej typu String z paczki std_msgs
8 # msg oznacza, ze importowane sa typy wiadomosci uzywane przez topic
9 # srv odnosiloby sie do typow wiadomosci uzywanych przez serwisy
10 from std_msgs.msg import String
11
12 def talker():
13     # Deklaracja, ze node 'talker' bedzie publikowal wiadomosc
14     # na topicu 'chatter' typu String
15     pub = rospy.Publisher('chatter', String, queue_size=10)
16     rospy.init_node('talker', anonymous=True)
17
18     rate = rospy.Rate(0.5) # 0.5hz czestotliwosc z jaka publikowane sa wiadomosci
19     while not rospy.is_shutdown():
20         hello_str = "hello world %s" % rospy.get_time()
21         rospy.loginfo(hello_str)
22         pub.publish(hello_str) #publikowanie wiadomosci
23         rate.sleep()
24
25 if __name__ == '__main__':
26     try:
27         talker()
28     except rospy.ROSInterruptException:
29         pass
```

```
1 #!/usr/bin/env python
2 import rospy
3 from geometry_msgs.msg import Pose2D
```

```
1 def pub_pose():
2     pub = rospy.Publisher('test_pose', Pose2D, queue_size=10)
3     rospy.init_node('pose_publisher', anonymous=True)
4     pose = Pose2D()
5     pose.x = 3
6     pose.y = 0
7     pose.theta = 1
8
9     rate = rospy.Rate(0.5) # 0.5Hz
10    while not rospy.is_shutdown():
11        pub.publish(pose)
12        rate.sleep()
```

```
1 if __name__ == '__main__':
2     try:
3         pub_pose()
4     except rospy.ROSInterruptException:
5         pass
```

Zadanie 1

Na podstawie przykładów należy napisać własnego Publisher'a, który będzie odpowiedzialny za publikowanie prędkości cmd_vel żółwia. W tym celu należy zaimportować odpowiednią bibliotekę, która zawiera typ publikowanej wiadomości. Wiadomość jest typu Twist i dla prędkości liniowej w polu x należy wpisać prędkość postępową, natomiast dla prędkości obrotowej w polu z prędkość obrotową.

Nowy Subscriber

Zapoznać się z kodem przedstawionym w poniższych przykładach odpowiadającym za stworzenie nowego Subscriber'a. Poniższe przykładowe programy dostępne są w folderze ROS_example (pliki sub_example_1.ipynb oraz sub_example_2.ipynb). Do zadziałania pierwszego subscriber'a konieczne jest uruchomienie pierwszego przykładowego publisher'a odpowiedzialnego za publikowanie wiadomości na topicu chatter.

```
1 #!/usr/bin/env python
2 import rospy
3 from std_msgs.msg import String
```

```
1 def callback(data):
2     # data - zmienna przechowująca odebrana wiadomosci typu String.
3     rospy.loginfo(rospy.get_caller_id() + "I heard %s", data.data)
4
5 def listener():
6     # In ROS, nodes are uniquely named. If two nodes with the same
7     # name are launched, the previous one is kicked off. The
8     # anonymous=True flag means that rospy will choose a unique
9     # name for our 'listener' node so that multiple listeners can
10    # run simultaneously.
11    rospy.init_node('listener', anonymous=True)
12
13    # Odczyt publikowanej wiadomosci na topicu 'chatter' typu String
14    # i wywołanie funkcji callback
15    rospy.Subscriber("chatter", String, callback)
16
17    # spin() simply keeps python from exiting until this node is stopped
18    rospy.spin()
```

```
1 if __name__ == '__main__':
2     listener()
```

```
1 #!/usr/bin/env python
2 import rospy
3 from turtlesim.msg import Color
```

```
1 def callback(data):
2     rospy.loginfo("r = %s, g = %s, b = %s", data.r, data.g, data.b)
3
4 def print_color():
5     rospy.init_node('color_listener', anonymous=True)
6     rospy.Subscriber("/turtle1/color_sensor", Color, callback)
7     rospy.spin()
```

```
1 if __name__ == '__main__':
2     print_color()
```

Zadanie 2

Na podstawie przykładów należy napisać własnego Subscriber'a, który będzie odpowiedzialny za odbieranie aktualnego położenia żółwia. W tym celu należy zaimportować odpowiednią bibliotekę, która

zawiera typ odbieranej wiadomości. Należy wyświetlić odebrane położenie x,y oraz kąt theta przeliczony z radianów na stopnie. Do konwersji należy użyć metody `math.degrees` z biblioteki `math`.

Tworzenie Subscriber'a i Publisher'a w jednym skrypcie

Zapoznać się z kodem przedstawionym w poniższym przykładzie odpowiadającym za stworzenie nowego Publisher'a. Poniższe przykładowe programy dostępne są w folderze `ROS_example` (pliki `subscribe_and_publish_example_p1.ipynb` oraz `subscribe_and_publish_example_p2.ipynb`).

Publisher dostarcza wiadomość typu `Int32` do utworzonego publisher'a i subscriber'a w jednym nodzie.

```
In [1]: #!/usr/bin/env python
#Przykładowy publisher dostarczający wiadomości do drugiego skrypty
import bibliotek i wiadomości rosowych
import rospy
from std_msgs.msg import Int32
```

```
In [2]: def pub_old():
    pub = rospy.Publisher('old_int', Int32, queue_size=10)
    rospy.init_node('old_pub', anonymous=True)
    oldInt = Int32()
    oldInt.data = 13

    rate = rospy.Rate(1) # 1hz
    while not rospy.is_shutdown():
        pub.publish(oldInt)
        rate.sleep()
```

```
In [ ]: if __name__ == '__main__':
    try:
        pub_old()
    except rospy.ROSInterruptException:
        pass
```

Poniżej przedstawiony jest przykładowy program odbierający wiadomość `/old_int` typu `Int32` i publikujący nową wiadomość `/new_int` typu `Int32`.

```
In [1]: #!/usr/bin/env python
#jednoczesny publisher i subscriber
import bibliotek i wiadomości rosowych
import rospy
from std_msgs.msg import Int32
```

```
In [2]: oldInt=Int32()

def fnc_callback(msg):
    global oldInt
    oldInt = msg
```

```
In [ ]: if __name__ == '__main__':
    rospy.init_node('NODE_SUB_AND_PUB')

    sub=rospy.Subscriber('/old_int', Int32, fnc_callback)
    pub=rospy.Publisher('/new_int', Int32, queue_size=1)
    rate=rospy.Rate(1)
    newInt = Int32()

    while not rospy.is_shutdown():
        newInt = oldInt
        if oldInt.data >= 10:
            #ograniczenie wartosci do 10
            newInt.data = 10
        pub.publish(newInt)
        rate.sleep()
```

Wynik działania powyższych skryptów można zaobserwować w terminalu dokonując podglądu odpowiednich topiców (`/new_int`, `/old_int`), a także można wyświetlić sobie graf z zależnościami pomiędzy nimi (`rqt_graph`).

Zadanie 3

Wzoruując się na powyższym przykładzie należy utworzyć własny plik i zmodyfikować powyższy kod tak, aby odbierać wiadomość o położeniu żółwia (`/turtle1/pose`). Ponadto jednocześnie ma być publikowana wiadomość dotycząca prędkości (`/turtle1/cmd_vel`). Jeżeli położenie x jest z przedziału `<4,6>` to prędkość obrotowa żółwia powinna być o połowę mniejsza. Prędkości postępowe i obrotowe można ustalić dowolnie

dobierając je doświadczalnie tak, by przy dłuższym uruchomieniu wykonywany ruch był powtarzalny oraz by żółw nie zatrzymywał się na obrzeżach.